

# Turbocharge Your Playwright

Features you can ship next Monday

Kevin Roe · QA or the Highway 2026

# Who this is for

Already writing Playwright tests and want to level up? Right room.

Not a beginner intro. Not a framework deep-dive.  
High-leverage features most teams walk right past.

# Under the hood: DevTools Protocol

*Playwright doesn't talk to the browser through WebDriver. It talks through **DevTools Protocol** — the same channel Chrome DevTools uses. That's why downloads, network interception, console events, and trace recording are first-class APIs here and not in your Selenium suite.*

Selenium 4 added a CDP escape hatch — Chrome-only, outside the WebDriver standard.

# Today's Agenda

- Act 1 — **Type Less. Test Better.**
- Act 2 — **Frontend without a backend**
- Act 3 — **Backend through the browser**
- Act 4 — **Debugging and observability**
- Closer — **What you can build with Playwright**

**Monday takeaway** → Every feature passes the "could you ship this next Monday?" test.



# Type Less. Test Better.

---

Reduce the typing and maintenance burden.  
Every feature is a one-liner add to an existing suite.

# addLocatorHandler()

## PAIN

### TYPESCRIPT

```
// Cookie banner blocks every test
await page.getByText('Accept all').click();
await page.getByRole('button', { name: 'Subscribe' })
  .click(); // fails if banner reappears
```

### JAVA

```
// Cookie banner blocks every test
page.getText("Accept all").click();
page.getByRole(BUTTON, name("Subscribe"))
  .click(); // fails if banner reappears
```

One new popup = copy-pasting dismiss logic into every test that touches that page.

# addLocatorHandler()

## TYPESCRIPT

```
await page.addLocatorHandler(  
  page.getByText('Accept all'),  
  async (locator) => await locator.click()  
);  
// runs on every navigation, automatically
```

## JAVA

```
page.addLocatorHandler(  
  page.getText("Accept all"),  
  locator -> locator.click()  
);  
// runs on every navigation, automatically
```

**Monday takeaway** → One line. Stack multiple handlers for sites with several popups.

# Locator composition

PAIN

TYPESCRIPT

```
await page.locator(  
  'table tr:nth-child(3) td:nth-child(2) button')  
  .click();
```

JAVA

```
page.locator(  
  "table tr:nth-child(3) td:nth-child(2) button")  
  .click();
```

Your selector knows the DOM structure. The frontend team wraps a `<div>` in a `<section>` and 30 tests break.

# Locator composition

## TYPESCRIPT

```
await page.getByRole('row')  
  .filter({ hasText: 'Widget A' })  
  .getByRole('button', { name: 'Delete' })  
  .click();
```

## JAVA

```
page.getByRole(ROW)  
  .filter(new Locator.FilterOptions()  
    .setHasText("Widget A"))  
  .getByRole(BUTTON,  
    new Page.GetByRoleOptions()  
      .setName("Delete"))  
  .click();
```

`filter()` · `and()` · `or()` · `nth()` · `first()` · `last()`

**Monday takeaway** → Reads like a sentence. Survives refactors that break CSS selectors.

# Web-first assertions vs.

## Thread.sleep

PAIN

TYPESCRIPT

```
await saveButton.click();  
await page.waitForTimeout(2000); // 🙌  
expect(await successBanner.isVisible()).toBe(true);  
// Jest/Vitest – point-in-time, no retry
```

JAVA

```
saveButton.click();  
Thread.sleep(2000); // 🙌  
assertTrue(successBanner.isVisible());  
// JUnit – point-in-time, no retry
```

Too short = flaky. Too long = slow. Usually both in the same suite.

# Web-first assertions vs.

## Thread.sleep

### TYPESCRIPT

```
await saveButton.click();  
await expect(successBanner)  
  .toBeVisible(); // retries until true  
// The assertion IS the wait.
```

### JAVA

```
saveButton.click();  
assertThat(successBanner)  
  .isVisible(); // retries until true  
// The assertion IS the wait.
```

**Monday takeaway** → Replace every sleep with the assertion itself. Faster AND flake-resistant.

# page.waitForResponse()

PAIN

TYPESCRIPT

```
await saveButton.click();  
await page.waitForTimeout(1500); // hoping API responded  
await expect(page.getByText('Saved')).toBeVisible();
```

JAVA

```
saveButton.click();  
Thread.sleep(1500); // hoping API responded  
assertThat(page.getByText("Saved")).isVisible();
```

You're guessing how long the API takes. The right number today is wrong on a slow CI box tomorrow.



# page.waitForResponse()

## TYPESCRIPT

```
const resp = await page.waitForResponse(
  '**/api/save',
  async () => await saveButton.click()
);
// returns the moment the response arrives
expect(resp.status()).toBe(200);
```

## JAVA

```
Response resp = page.waitForResponse(
  "**/api/save",
  () -> saveButton.click()
);
// returns the moment the response arrives
assertThat(resp.status()).isEqualTo(200);
```

**Monday takeaway** → Same "wait for the actual signal" mindset — applied to network instead of DOM.

# File downloads

## TYPESCRIPT

```
const dl = await page.waitForEvent(
  'download',
  async () => await exportButton.click());
await dl.saveAs('report.csv');
// now assert on the file
```

## JAVA

```
Download dl = page.waitForDownload(
  () -> exportButton.click());
dl.saveAs(Paths.get("report.csv"));
// now assert on the file
```

**Monday takeaway** → Selenium-trained testers skip download tests because they assume they're hard. They're not.

# File uploads

## TYPESCRIPT

```
await page.locator('input[type=file]')  
  .setInputFiles('test-data/photo.jpg');
```

## JAVA

```
page.locator("input[type=file]")  
  .setInputFiles(  
    Paths.get("test-data/photo.jpg"));
```

**Monday takeaway** → No real file picker needed. Works headless in CI.

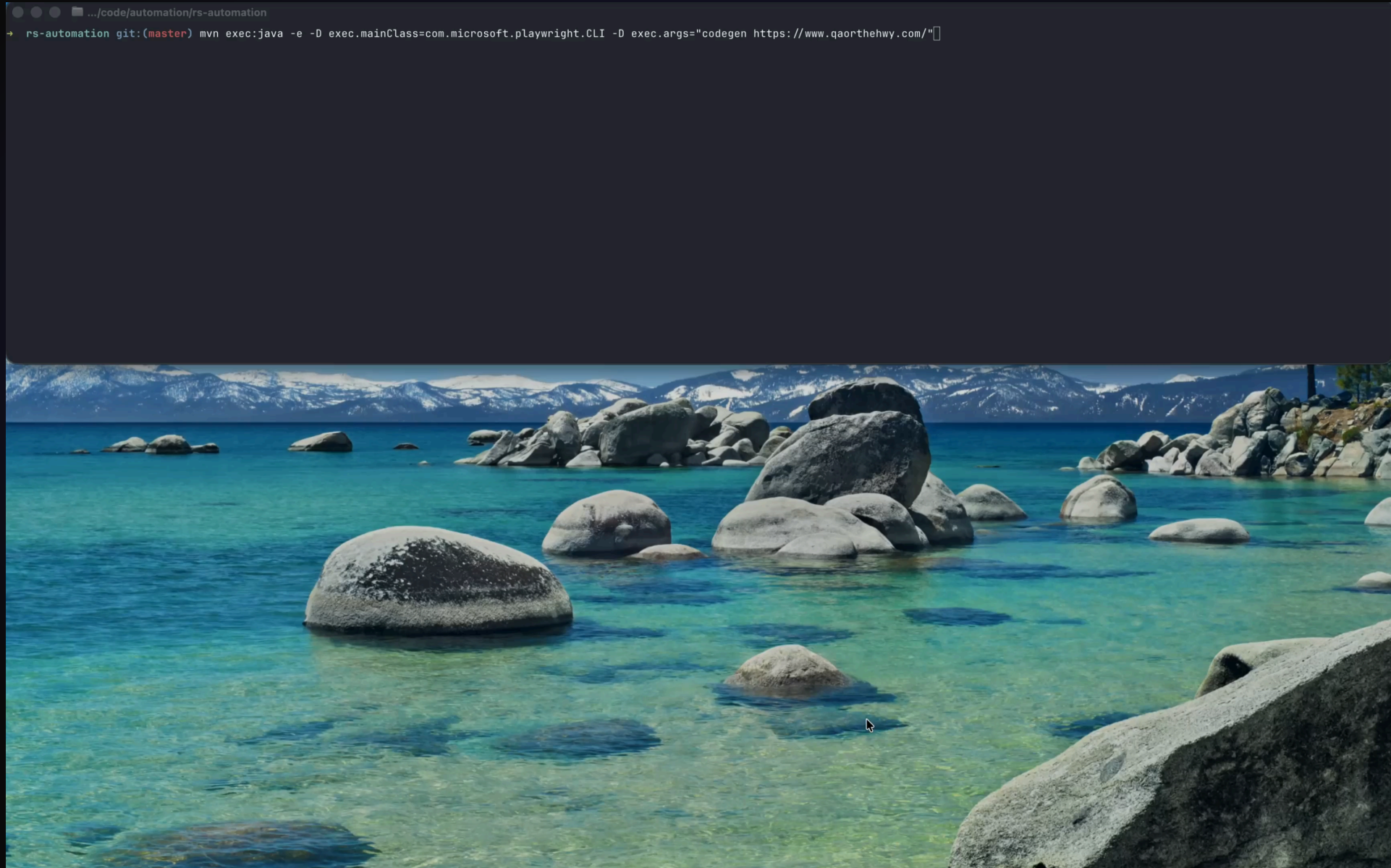
# Are you using Codegen?

Most teams don't.

It writes the tests you'd want to write — no AI needed!

# Codegen

ACT 4



# Frontend Without a Backend

---

Test UI behavior without a working backend.  
Mock it, record it, freeze it, disconnect it.

# page.route()

## Mock your backend calls

### TYPESCRIPT

```
await page.route('**/api/products',  
  route => route.fulfill({  
    status: 500,  
    body: 'Internal Server Error'  
  }));  
// Also: abort(), continue() with overrides
```

### JAVA

```
page.route("**/api/products",  
  route -> route.fulfill(  
    new Route.FulfillOptions()  
      .setStatus(500)  
      .setBody("Internal Server Error")));  
// Also: abort(), continue() with overrides
```

**Monday takeaway** → Test every backend failure state without touching the backend.

# routeFromHAR()

## Replay tests against a recorded backend

### TYPESCRIPT

```
// Record once:
const context = await browser.newContext({
  recordHar: { path: 'fixtures/api.har' }
});
// Replay forever:
await context.routeFromHAR('fixtures/api.har',
  { notFound: 'fallback' });
```

### JAVA

```
// Record once:
context = browser.newContext(
  new Browser.NewContextOptions()
    .setRecordHarPath(Paths.get("fixtures/api.har")));
// Replay forever:
context.routeFromHAR(Paths.get("fixtures/api.har"),
  new BrowserContext.RouteFromHAROptions()
    .setNotFound(RouteFromHARNotFound.FALLBACK));
```

**Monday takeaway** → Commit the HAR. Your backend becomes optional for UI tests.



# page.clock

## Control time in your tests

### TYPESCRIPT

```
// Freeze time at a specific moment
await page.clock.setFixedTime(
  new Date('2024-12-31T23:59:55Z'));
await page.goto('https://example.com/countdown');

// Fast-forward 10 seconds
await page.clock.fastForward(10_000);
```

### JAVA

```
// Freeze time at a specific moment
page.clock().setFixedTime(
  Instant.parse("2024-12-31T23:59:55Z"));
page.navigate("https://example.com/countdown");

// Fast-forward 10 seconds
page.clock().fastForward(10_000);
```

**Monday takeaway** → Test session expiry, countdowns, and scheduled jobs without waiting.

# context.setOffline(true)

## Test your offline experience

### TYPESCRIPT

```
await context.setOffline(true);
await page.reload();
await expect(page.getByText("You're offline"))
  .toBeVisible();

await context.setOffline(false);
await expect(page.getByText('Back online'))
  .toBeVisible();
```

### JAVA

```
context.setOffline(true);
page.reload();
assertThat(page.getByText("You're offline"))
  .isVisible();

context.setOffline(false);
assertThat(page.getByText("Back online"))
  .isVisible();
```

Monday takeaway → Offline UX is one line away. Test reconnection flows in CI.

# Environment emulation

Any device, locale, or timezone — no device farm needed

## TYPESCRIPT

```
const ctx = await browser.newContext({  
  viewport: { width: 390, height: 844 },  
  userAgent: 'Mozilla/5.0 (iPhone ...)',  
  locale: 'ja-JP',  
  timezoneId: 'Asia/Tokyo',  
  geolocation: { latitude: 35.6595, longitude: 139.7004 },  
  colorScheme: 'dark',  
});  
await ctx.grantPermissions(['geolocation']);
```

## JAVA

```
BrowserContext ctx = browser.newContext(  
  new Browser.NewContextOptions()  
    .setViewportSize(390, 844)  
    .setUserAgent("Mozilla/5.0 (iPhone ..."  
    .setLocale("ja-JP")  
    .setTimezoneId("Asia/Tokyo")  
    .setGeolocation(new Geolocation(35.6595,  
    .setColorScheme(ColorScheme.DARK));  
ctx.grantPermissions(List.of("geolocation"));
```

Monday takeaway → No Appium. No device farm. No separate suite.

# Backend Through the Browser

---

Flip the direction — use Playwright to reach the backend directly.

Real auth, real session, no UI setup chain.

*"Your frontend is the most authoritative documentation your backend has — and Playwright lets you read it at runtime."*

# API-seeded fixtures

ACT 4

## Skip the UI setup chain

### TYPESCRIPT

```
const api = await request.newContext({
  baseURL: process.env.BASE_URL
});

await api.post('/api/users', {
  data: { email: 'test@example.com', plan: 'enterprise' }
});

await page.goto('/dashboard');
```

### JAVA

```
APIRequestContext api = playwright.request()
    .newContext(new APIRequest.NewContextOptions()
        .setBaseURL(System.getenv("BASE_URL")))

api.post("/api/users",
    RequestOptions.create().setData(Map.of(
        "email", "test@example.com",
        "plan", "enterprise")));

page.navigate("/dashboard");
```

**Monday takeaway** → Skip the UI setup chain. Tests that took 30s of clicks take 200ms.

# Reuse your session across tests

Log in once. Never log in again.

## TYPESCRIPT

```
// Global setup – log in, then save:  
await context.storageState({ path: 'auth.json' });  
  
// Every test – start already authenticated:  
const context = await browser.newContext({  
  storageState: 'auth.json'  
});
```

## JAVA

```
// Global setup – log in, then save:  
context.storageState(new BrowserContext.StorageStateOptions()  
    .setPath(Paths.get("auth.json")));  
  
// Every test – start already authenticated:  
BrowserContext context = browser.newContext(  
    new Browser.NewContextOptions()  
        .setStorageStatePath(Paths.get("auth.json")));
```

**Monday takeaway** → Log in once per suite run. Every test starts already authenticated.

# Layer-crossing auth

Browser login → scrape what you need → call the backend directly

## TYPESCRIPT

```
// Scrape CSRF and session cookie
const csrf = await page.evaluate(() => document.querySelector('meta[name="csrf-token"]').content);
const session = (await context.cookies()).find(c => c.name === 'session');

// Set CSRF and Session on the api context
const api = await request.newContext({
  baseURL: process.env.BASE_URL,
  extraHTTPHeaders: { 'X-CSRF-Token': csrf, 'Cookie': `session=${session.value}` }
});

// Hit the backend directly – no more UI
const resp = await api.get('/api/account/settings');
expect(resp.status()).toBe(200);
```

# Layer-crossing auth

ACT 4

Browser login → scrape what you need → call the backend directly

## JAVA

```
// Scrape CSRF and session cookie
String csrf = (String) page.evaluate("document.querySelector('meta[name=\"csrf-token\"]').content");
Optional<Cookie> session = context.cookies().stream()
    .filter(c -> c.name.equals("session")).findFirst().orElseThrow();

// Set CSRF and Session on the api context
APIRequestContext api = playwright.request().newContext(
    new APIRequest.NewContextOptions()
        .setBaseURL(System.getenv("BASE_URL"))
        .setExtraHTTPHeaders(Map.of("X-CSRF-Token", csrf, "Cookie", "session=" + session.value)));

// Hit the backend directly – no more UI
APIResponse resp = api.get("/api/account/settings");
assertThat(resp.status()).isEqualTo(200);
```



*Postman can't do this. Curl can't do this.  
REST Assured can't do this without a separate browser  
automation tool like Selenium.*

*Playwright can — because it's the only tool in your test  
stack with a real browser and a real HTTP client in the  
same process.*

# Debugging & Observability

---

When a test fails — what do you see?

*"Playwright remembers everything so you don't have to."*

# Trace Viewer

## TYPESCRIPT

```
await context.tracing.start({
  screenshots: true,
  snapshots: true,
  sources: true
});

// ... run your test ...

await context.tracing.stop({ path: 'trace.zip' });
```

## JAVA

```
context.tracing().start(
    new Tracing.StartOptions()
        .setScreenshots(true)
        .setSnapshots(true)
        .setSources(true));

// ... run your test ...

context.tracing().stop(new Tracing.StopOptions()
    .setPath(Paths.get("trace.zip")));
```

**Monday takeaway** → Open at `trace.playwright.dev` — no install, send a URL to a colleague.

# Trace Viewer — Actions

ACT 4

The screenshot displays the Playwright Trace Viewer interface. The top bar shows the browser tab 'Playwright Trace Viewer' and the URL 'localhost:56296/trace/index.html?trace=http://localhost:56296/data/80c8f6740da05127fe2dfa51e725b489ce65d129.zip'. Below the bar, the test path is 'demo-todo-app.spec.ts:60 > New Todo > should append new items to the bottom of the list'. A timeline at the top shows the test duration from 0 to 1.0s, with a yellow bar indicating the active test step.

The main content area is divided into two panels. The left panel, titled 'Actions', lists the test steps and their durations:

- > Before Hooks 797ms
- locator.fill getByPlaceholder... 23ms
- locator.press getByPlacehol... 41ms
- locator.fill getByPlaceholder('... 9ms
- locator.press getByPlacehold... 8ms
- locator.fill getByPlaceholder('... 9ms
- locator.press getByPlacehold... 9ms
- expect.toBeVisible getByText... 4ms
- expect.toHaveText getByTest... 4ms
- expect.toContainText getByT... 3ms
- expect.toHaveText getByTestl... 2ms
- expect.toHaveText getByTestl... 2ms
- page.waitForFunction 8ms
- > After Hooks 0ms

The right panel, titled 'Action Before After', shows a preview of the application state during the test. It displays a web browser window with the URL 'https://demo.playwright.dev/todomvc/#/'. The page shows a 'todos' application with a list of items: 'buy some cheese' and 'feed the cat'. The status bar at the bottom indicates '2 items left' and 'All Active Completed'.

The bottom of the interface features a navigation bar with tabs: 'Locator', 'Call', 'Log', 'Errors', 'Console' (1), 'Network' (6), 'Source', and 'Attachments'.

# Trace Viewer — Screenshots

ACT 4

The screenshot displays the Playwright Trace Viewer interface. At the top, the browser address bar shows the URL: `localhost:9323/trace/index.html?trace=http://localhost:9323/data/06fc3bbeaa4e94068c51e79a04cb0b77f5d5f091.zip`. The main header indicates the test run: **Playwright** demo-todo-app.spec.ts:108 > Mark all as completed > complete all checkbox should update state when items are completed /

The central timeline shows a sequence of actions over 700ms. A specific action, `locator.check getByLabel('Mark all as complete')`, is highlighted with a duration of 64ms. A detailed view of this action is shown on the right, featuring a screenshot of the 'todos' application interface. The interface includes a title 'What needs to be done?', a list of tasks (buy some cheese, feed the cat, book a doctors appointment), and a status bar showing '3 items left' and buttons for 'All', 'Active', and 'Completed'.

On the left, the 'Actions' panel lists the following steps:

- Before Hooks: 506ms
- beforeEach hook: 111ms
- locator.press `getByPlac...`: 9ms
- locator.fill `getByPlaceho...`: 8ms
- locator.press `getByPlac...`: 7ms
- page.waitForFunction: 9ms
- locator.check `getByLabel('...`: 64ms
- expect.toBeChecked `getByL...`: 2ms

At the bottom, the 'Call' tab is selected, showing the following details for the `locator.press` action:

- TIME**
  - wall time: 18/09/2023, 16:14:05
  - duration: 9ms
- PARAMETERS**
  - locator: `getByPlaceholder('What needs to be done?')`
  - strict: `true`

# Trace Viewer — Errors

ACT 4

Playwright Trace Viewer

localhost:9323/trace/index.html?trace=http://localhost:9323/data/a4b1e03052e0a7ec12b32bd097b2e9d5fd343dd9.zip

demo-todo-app.spec.ts:298 > Clear completed button > should remove completed items when clicked

0 500ms 1.0s 1.5s 2.0s 2.5s 3.0s 3.5s 4.0s 4.5s 5.0s 5.5s

Actions Metadata

- > Before Hooks 389ms
  - locator.check getByTestId('t... 20ms
  - locator.click getByRole('but... 66ms
  - expect.toHaveCount getByTe... 5.0s
- > After Hooks 0ms

Action Before After

480ms expect.toHaveCount getByTestId('todo-item') 5.0s

Timed out 5000ms waiting for expect(received).toHaveCount(expected) // deep equality

Expected: 3  
Received: 2

Call log:

- expect.toHaveCount with timeout 5000ms
- waiting for getByTestId('todo-item')
- locator resolved to 2 elements
- unexpected value "2"

# Trace Viewer — Console

ACT 4

The screenshot displays the Playwright Trace Viewer interface. At the top, the browser address bar shows the URL: `localhost:9323/trace/index.html?trace=http://localhost:9323/data/c6dbc3ef1d6e375cde97fef22ac5a9764a66e607.zip`. The main header indicates the test file `demo-todo-app.spec.ts:181` and the specific test `Item > should allow me to edit an item`.

The central part of the interface features a timeline from 0 to 750ms. Below the timeline, the 'Actions' tab is active, listing a series of actions with their durations:

- > Before Hooks: 502ms
- locator.fill getByPlaceholder...: 17ms
- locator.press getByPlacehol...: 35ms
- locator.fill getByPlaceholder(...): 8ms
- locator.press getByPlacehold...: 7ms
- locator.fill getByPlaceholder(...): 8ms
- locator.press getByPlacehold...: 6ms
- locator.dblclick getByTestId...: 53ms
- expect.toHaveValue getByTe...: 6ms
- locator.fill getByTestId('todo-...': 7ms
- locator.press getByTestId('t...': 10ms
- expect.toHaveText getByTest...: 3ms
- page.waitForFunction: 9ms
- ✓ After Hooks: 0ms
- fixture: page: 0ms
- fixture: context: 0ms

To the right of the actions list, a preview window shows the application state at the time of the selected action. It displays a 'todos' application with a list of items: 'buy some cheese', 'buy some sausages', and 'book a doctors appointment'. The 'Active' filter is selected, and there are 3 items left.

At the bottom, the 'Console' tab is active, showing three log entries:

- 469ms: **Download the React DevTools for a better development experience: <https://fb.me/react-devtools>** (bundle.js:20881)
- 700ms: testing console logs
- 700ms: another console log from the test

# Trace Viewer — Network

ACT 4

Playwright Trace Viewer

localhost:9323/trace/index.html?trace=http://localhost:9323/data/c6dbc3ef1d6e375cde97fef22ac5a9764a66e607.zip

demo-todo-app.spec.ts:181 > Item > should allow me to edit an item

0 50ms 100ms 150ms 200ms 250ms 300ms 350ms 400ms 450ms 500ms 550ms 600ms 650ms 700ms 750ms

Actions Metadata

> Before Hooks 502ms

- locator.fill getByPlaceholder... 17ms
- locator.press getByPlacehol... 35ms
- locator.fill getByPlaceholder(... 8ms
- locator.press getByPlacehold... 7ms
- locator.fill getByPlaceholder(... 8ms
- locator.press getByPlacehold... 6ms
- locator.dblclick getByTestId... 53ms
- expect.toHaveValue getByTe... 6ms
- locator.fill getByTestId('todo-... 7ms
- locator.press getByTestId('t... 10ms
- expect.toHaveText getByTest... 3ms
- page.waitForFunction 9ms

✓ After Hooks 0ms

- fixture: page 0ms
- fixture: context 0ms

Action Before After

https://demo.playwright.dev/todomvc

Locator Call Log Errors Console 3 Network 6 Source Attachments

|  | Status    | Method | Request      | Content Type          | Duration | Size   | Route |
|--|-----------|--------|--------------|-----------------------|----------|--------|-------|
|  | 408ms 200 | GET    | /bundle.js   | application/javasc... | 62ms     | 238.0K |       |
|  | 395ms 200 | GET    | /director.js | application/javasc... | 19ms     | 6.0K   |       |
|  | 397ms 200 | GET    | /index.css   | text/css              | 21ms     | 2.3K   |       |
|  | 398ms 200 | GET    | /base.css    | text/css              | 22ms     | 947    |       |
|  | 373ms 200 | GET    | /            | text/html             | 17ms     | 705    |       |
|  | 356ms 301 | GET    | /todomvc     | text/html             | 160ms    | -      |       |



# `page.pause()` + Playwright Inspector

## TYPESCRIPT

```
// Drop this anywhere in a failing test
await page.pause();
// → Inspector opens, test freezes
```

```
# Or, from the command line directly
PWDEBUG=1 npx playwright test my-test.spec.ts
```

## JAVA

```
// Drop this anywhere in a failing test
page.pause();
// → Inspector opens, test freezes
```

```
# Or, from the command line directly
PWDEBUG=1 mvn test -Dtest=MyFailingTest
```

Step through actions · Inspect live DOM · Run JS · Pick locators

Monday takeaway → Trace Viewer = after-the-fact. Inspector = in-the-

# Playwright Inspector

ACT 4

Step through, inspect the DOM, run JS, pick locators — all against a live paused test.

Playwright Inspector

Record Target: example.spec.ts

```
1 import { test, expect } from '@playwright/test';
2
3 test('has title', async ({ page }) => {
4   await page.goto('https://playwright.dev/');
5
6   // Expect a title "to contain" a substring.
7   await expect(page).toHaveTitle(/Playwright/);
8 });
9
10 test('get started link', async ({ page }) => {
11   await page.goto('https://playwright.dev/');
12   // Click the get started link.
13   await page.getByRole('link', { name: 'Get started' }).click();
14
15   // Expects the URL to contain intro.
16   await expect(page).toHaveURL(/.*intro/);
17 });
```

Pick locator `getByRole('link', { name: 'Get started' })`

> browserContext.newPage ✓ — 222ms

> page.goto(https://playwright.dev/) ✓ — 405ms

▼ `page.getByRole('link', { name: 'Get started' }).click()` II

- waiting for getByRole('link', { name: 'Get started' })
- locator resolved to visible `<a href="/docs/intro" class="getStarted_Sjon">Get started</a>`
- attempting click action
- waiting for element to be visible, enabled and stable
- element is visible, enabled and stable
- scrolling into view if needed

# Visual regression — TypeScript

Save the baseline

```
// First run — saves baseline to disk
await page.screenshot({
  path: 'snapshots/checkout.png'
});
```

Assert against it

```
// Built in to @playwright/test
await expect(page).toHaveScreenshot(
  'checkout.png');
// Subsequent: pixel-diff
// On failure: expected/actual/diff triple
```

**Monday takeaway** → `toHaveScreenshot()` handles baseline management automatically — built in to `@playwright/test`.

# Visual regression — Java

## Save the baseline

```
// First run – saves baseline to disk
page.screenshot(new Page.ScreenshotOptions()
    .setPath(Paths.get(
        "snapshots/checkout.png")));
```

## Assert against it

```
// ~60 lines + one image library
VisualAssert.assertScreenshot(
    page, "checkout");
// Subsequent: pixel-diff
// On failure: expected/actual/diff triple
```

Diff library: `com.github.romankh3 / image-comparison`

Full source: `github.com/kroe761/playwright-demos/tree/main/vrt` (plus Python and C#)

**Monday takeaway** → Works in Java today. Grab the ~60 lines of code from the repo.

# Observer patterns

ACT 4

## TYPESCRIPT

```
// JS console errors
page.on('console', msg => {
  if (msg.type() === 'error')
    errors.push(msg.text());
});

// Unhandled exceptions
page.on('pageerror', err =>
  failures.push(err.message));

// Failed network requests
page.on('requestfailed', req =>
  networkFailures.push(
    `${req.url()} - ${req.failure()}`));

expect(errors).toHaveLength(0);
```

## JAVA

```
// JS console errors
page.onConsoleMessage(msg -> {
  if (msg.type().equals("error"))
    errors.add(msg.text());
});

// Unhandled exceptions
page.onPageError(err ->
  failures.add(err.message()));

// Failed network requests
page.onRequestFailed(req ->
  networkFailures.add(
    req.url() + " - " + req.failure()));

assertThat(errors).isEmpty();
```

**Monday takeaway** → Catches "passing test, silently broken app." Wire or **ACT 4** benefit everywhere.

# traceparent — distributed tracing

## TYPESCRIPT

```
await context.setExtraHTTPHeaders({  
  traceparent: `00-${traceId}-${spanId}-01`  
});  
// Every request now carries this ID.  
// APM correlates test run to backend spans.
```

## JAVA

```
context.setExtraHTTPHeaders(Map.of(  
    "traceparent",  
    "00-" + traceId + "-" + spanId + "-01"  
));  
// Every request now carries this ID.  
// APM correlates test run to backend spans.
```



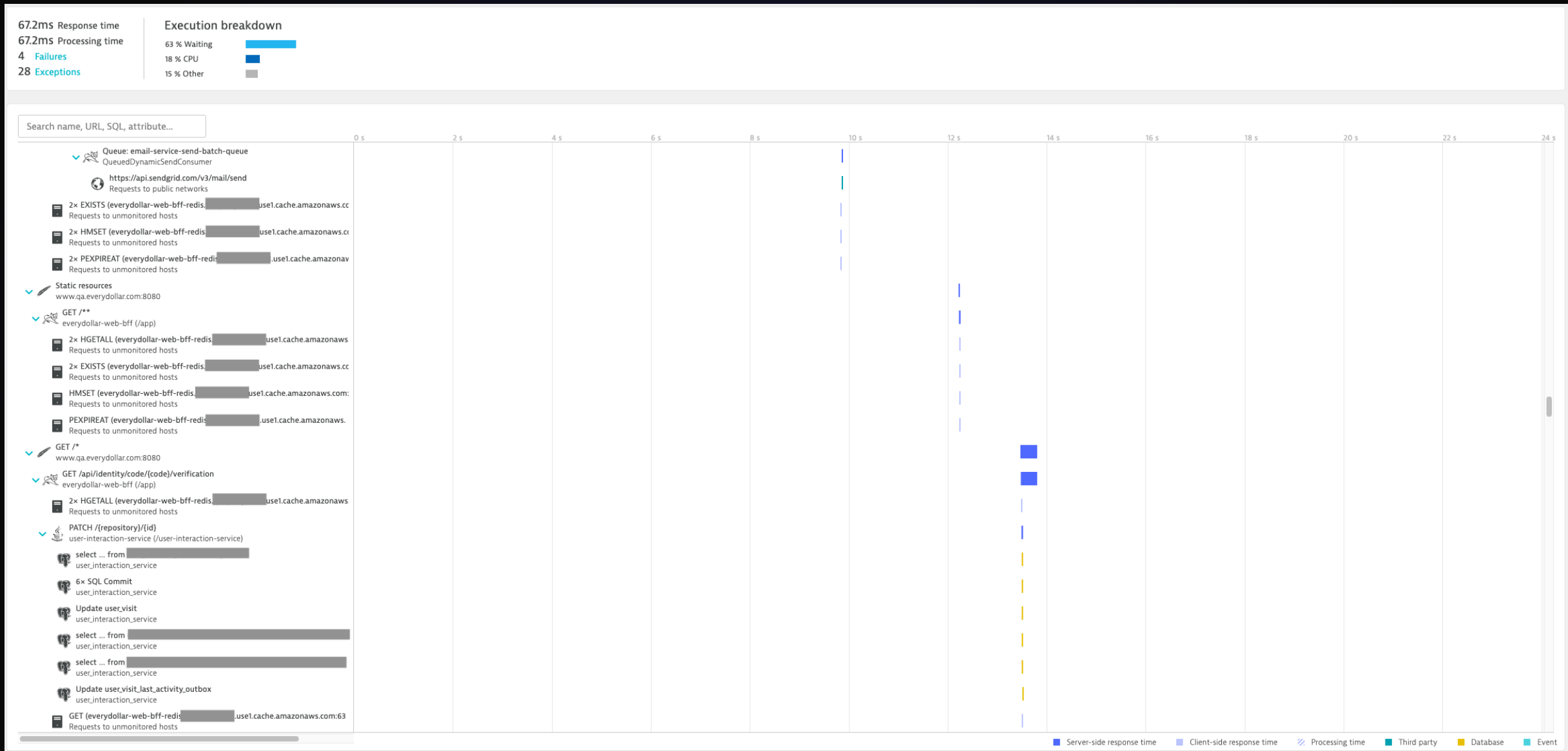
# The trace — beginning

ACT 4



# The trace — middle

ACT 4



# The trace — end

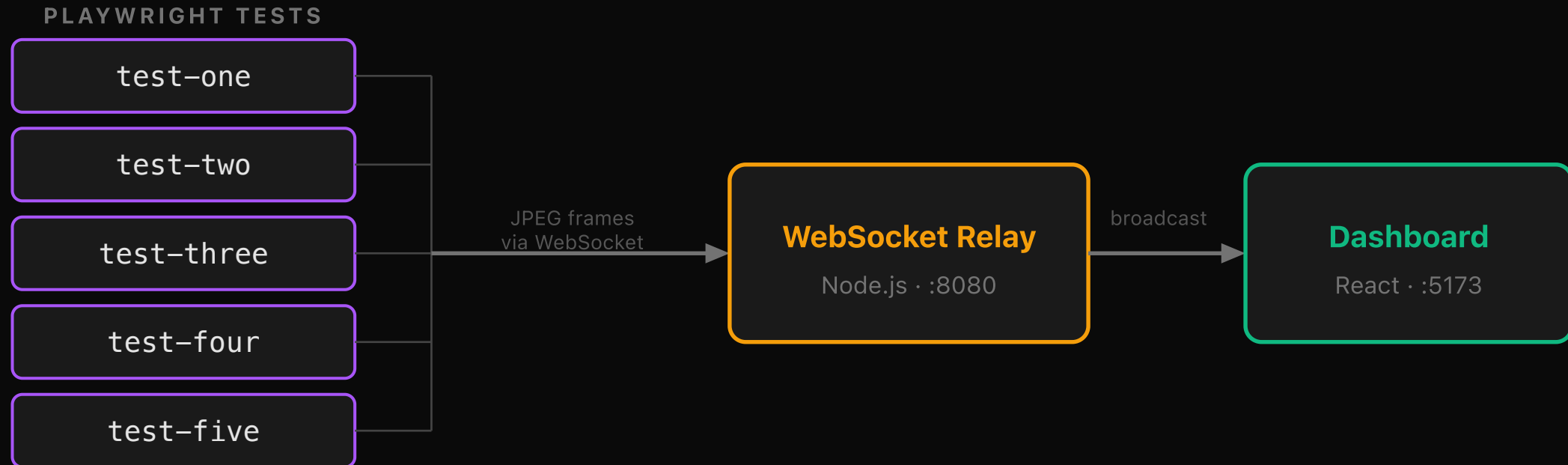


*Everything you've seen today — you can ship on Monday.*

*But here's one more thing you can do with Playwright.*

# Playwright CI Test Viewer

## Architecture



## Playwright Test Dashboard

0 running tests

Waiting for tests to start...

# The test page

**ThisIsATestPage!**

**0.5**

▶ 0:00 / 0:29



# Dashboard in action

## Playwright Test Dashboard

0 running tests

Waiting for tests to start...

```
→ screencast-dashboard git:(main) x ./run-tests.sh
```

▶ 0:00 / 0:54





# How it works

## TYPESCRIPT

```
// Open a websocket connection
const name = encodeURIComponent(testInfo.title);
const ws = new WebSocket(
  `ws://localhost:8080?name=${name}`
);

// Send the screencast to the websocket
await page.screencast.start({
  onFrame: (frame) => ws.send(frame.data),
  path: 'recording.webm',
  quality: 50,
  size: { width: 1280, height: 720 },
});
```

## JAVA

```
// Open a websocket connection
var url = "ws://localhost:8080?name=" + testName;
var ws = new WsClient(url);
ws.connect();

// Send the screencast to the websocket
page.screencast().start(
  new Screencast.StartOptions()
    .setOnFrame(f -> ws.send(f.getData()))
    .setPath(Paths.get("recording.webm"))
    .setQuality(50)
    .setSize(new ViewportSize(1280, 720))
);
```

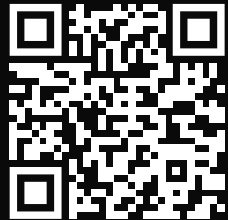
The repo sample: three processes · ~500 lines total · A few days of work.

*See exactly what every browser is doing in CI — live, while it's happening.*

# Questions?

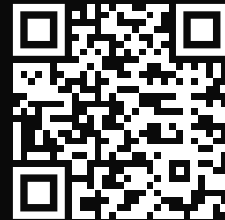
Also - what's the weirdest thing you've automated?

**Playwright Demos  
Repo**



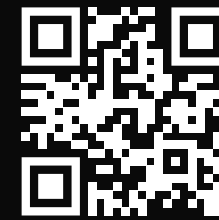
[github.com/kroe761/playwright-demos](https://github.com/kroe761/playwright-demos)

**LinkedIn**



[linkedin.com/in/kroe761](https://linkedin.com/in/kroe761)

**"Go Forth and  
Test" Blog**



[goforthandtest.com](https://goforthandtest.com)

**Session  
Feedback**



[QA or the Highway](#)

Go Forth and

*Automate!*

